

5. Authentication middleware

Maybe you've now got login working and users can sign up. Great! But part of an app is making sure that every "protected" page and every "API route" actually checks whether the user is logged in first before allowing them access.

A lot of vibe coded apps don't! And what happens is you end up with some pages that check auth and some that don't. E.g maybe your dashboard checks, but your settings page doesn't. And then an attacker will just go to your settings page and change whatever the heck they want (or run up your API bill lol).

You can avoid this by just ensuring middleware secures all routes.

4. Dependency/package security

When AI generates code, it will sometimes reference packages that don't exist. As in, your AI will just make up a package name. It happens less often now with good models, but still ~5% of AI-generated code contains references to non-existent packages.

The funny part is attackers know this. They will create malicious packages with those exact made-up names, and then when your AI goes to install the package, it installs their malware, which usually executes things like "give Nick all of your API keys".

Also: AI tends to reference older versions of libraries or use deprecated APIs. This means you might be pulling in packages with known security vulnerabilities that were patched months or years ago. So you must ensure you get updated packages as often as possible.



3. Server-side validation

Big issue AI does frequently: it puts **validation** on the frontend. If you didn't know what this meant, it does logic like "the form checks that the email is valid and the password is 8 characters" directly in the code that is exposed to users (which you can access in network requests).

Then on the server side it just trusts whatever comes in. Which is a glaring security flaw, since anyone with basic developer tools can bypass frontend validation entirely. They can open the browser console, call your API directly with curl, or use a tool like Postman.

Luckily, this is a similarly easy fix. You just tell your AI model to validate everything on the server. E.g if your app expects an email to sign it up, you just validate that it's actually an email on the **server**, not on the frontend of your app.

2. Row level security (RLS)

If you're using Supabase (which most vibe coders are rn), **Row Level Security** is probably the single most important security feature you need to add.

In simple terms, without RLS, anyone who has your Supabase URL and anon key (which are public, by design) can read, write, and delete every single row in your database. Just how Supabase works by default.

RLS on the other hand, is the thing that says "hey, User A can only see User A's data." Which makes sense. It isn't enabled by default for some reason, though imo it should be. But you can just turn this on and fix like 90% of errors.

PS—83% of exposed Supabase databases involve RLS misconfigurations. And AI tools will often create tables without enabling RLS, or enable it without actually adding policies, which makes every query return empty results and looks like a bug instead of a security issue. Luckily you can just click "enable" and it will be fine.

1. Environment variables and secret management

In a nutshell, lots of people will hardcode authentication tokens/secrets/environment variables into their code. This is kind of like putting your password on a sticky note and attaching it to your computer. Very dumb! Also, lots of hackers/etc will just go through your website/app and look for hardcoded mentions of "NEXT_PUBLIC_", which is a common string prefix before a key.

You don't do this anymore. Instead, you put them all in a file, like a .env, and then reference them using "imports". You also change the name of the code that imports any keys, to obfuscate and prevent people from running simple ctrl+f and finding sections of code that contain keys.

Anyway, the exact syntax is unimportant, but by doing it this way, you never have the actual key in plaintext in a file, which fixes a really low hanging fruit.

websites → app

The 80/20 of vibe coding security

~80% of security problems in vibe-coded apps come from five things:

1. Exposed environment variables and API keys.
2. Missing or broken Row Level Security (RLS) on your database.
3. No server-side validation (trusting the frontend for everything).
4. Using outdated or hallucinated packages.
5. Not having proper authentication middleware.

If you fix these five things, you are ahead of pretty much everyone vibe coding right now. It is not perfect (no security ever is) but it will allow you to launch apps without feeling like a fraud, or needlessly endangering people's credentials.